

Serverless Credit Card Fraud Risk Scoring on AWS: Design, Deployment, and Scalability Considerations

An end-to-end AI ecosystem implementation using Amazon Web Services (AWS) with an operational serverless inference fallback under sandbox constraints.

Name : Sharon Danish Laus

Revision Timestamp: 22-04-2026 21:48:49

Module Director: Patrick Denny
Teaching Assistant: Ankita Saini

Abstract

This report presents the design and implementation of a production-oriented AI ecosystem for credit card fraud risk scoring on Amazon Web Services (AWS). The business problem is to identify potentially fraudulent transactions quickly enough to support operational intervention while controlling false positives that can negatively affect customer experience. I selected this topic because fraud detection combines a challenging imbalanced machine learning classification problem with practical cloud engineering constraints, making it suitable both for academic assessment and for portfolio demonstration. The implemented ecosystem includes Amazon S3 for data and artifact storage, Amazon SageMaker notebook compute for model development, AWS Lambda for serverless inference logic, Amazon API Gateway for external API access, and Amazon CloudWatch for operational telemetry. A model based on XGBoost was trained and calibrated using threshold tuning for the positive class. Final model performance on the hold-out set was Precision 0.9302, Recall 0.8163, F1 0.8697, ROC-AUC 0.9774, and PR-AUC 0.8778 at an operating threshold of 0.87. During deployment in the AWS Academy sandbox, SageMaker endpoint provisioning failed due to environment constraints, so a serverless fallback path was implemented using Lambda plus API Gateway with S3-backed prediction artifacts. This adaptation preserved functional goals and enabled cloud KPI capture including request latency, invocation counts, and error rates. The resulting solution demonstrates not only ML performance, but also architectural decision making, resilience under platform limitations, and scalability planning aligned to AWS Well-Architected guidance.[1], [5]-[9]

Contents

Abstract.....	2
Introduction	3
AI Ecosystem Architecture Used	3
Model Description	5
Scalability Considerations	10
References	10
Figure 1: Architecture used to create the fraud risk scoring AI ecosystem on AWS (training in SageMaker notebook, serving via Lambda/API Gateway, monitoring via CloudWatch).....	5
Figure 2: Model Training Output	7
Figure 3:Threshold Tuning	7
Figure 4: Lambda Test Result	8
Figure 5: API Lambda Metrics	8
Figure 6: API Responses	8
Figure 7: Cloud Watch API metrics	9
Figure 8: Artifact Listing.....	9

Introduction

The problem of fraud detection in card payments is essentially a risk management one in the presence of uncertainty, and latency constraints, as well as with highly skewed class distributions. Majority of the transactions are valid and very small percentage of the transactions are fake. With this disparity, naive models may seem valid and yet fail to perform well on those events that have the most significance. In the case of an operational fraud team, the question of whether a model predicts correctly in aggregate is only practical to the extent that it can identify cases that are of high risk promptly and to an acceptable degree of accuracy to be useful in supporting intervention workflows, such as step-up authentication, temporary hold, or manual review queues. As an end-user, the system must be responsive as a risk service. In a near real time situation, it is expected to provide an interpretable score of fraud and decision label, given an incoming transaction situation. Such a response must be reliable, accessible and verifiable. Downstream applications, fraud analysts and operations teams are some of the users of the service. All groups have their own requirements: applications need low-latency APIs, analysts need stable logs and results, and operators of the platform need to observe and control costs. System-design wise, the issue is to overcome the bridge between the functional and non-functional requirements. The model should have effective fraud detection capabilities, non-functionally, the architecture should be secure, scalable and economical. In this report, the two layers are connected by considering the model to be just a part of a broader AI ecosystem, and not an experiment with a notebook. This architecture was created with the principles of modularity: S3 data storage and versioned artifacts, SageMaker notebook compute to model development, Lambda to serve inference lightweight, API Gateway to abstract the API, and CloudWatch to monitor. Its implementation was in an iterative manner. The first step was to train a baseline model and evaluate it. Second, class imbalance treatment and threshold tuning were used to maximize metrics that were based on fraud. Third, the components of serving and monitoring were incorporated to design a functional path towards inference. Fourth, with sandbox restrictions with managed endpoint provisioning, the architecture was modified to a serverless fallback without evidence quality loss. The very fact that this is a type of ecosystem adaptation is a significant ecosystem competency, since in real-life cloud projects, policy, budget, and quota restrictions is frequently encountered. Overall, the project shows a viable, full-fledged AI ecosystem in which the quality of models, the behavior of the services and cloud services are taken into account as a single entity. This framing is both in line with the expectations of the enterprise and the focus of the module in the field of theory and practice in advanced AI ecosystems. [2], [3], [9]

AI Ecosystem Architecture Used

The ecosystem architecture (Figure 1) can be interpreted as a series of five layers: data, modelling, serving, API access and operations. The design goal was to maintain clean lines among layers in order to be able to develop each separately. [17]

Data layer:

Amazon S3 is the long-lasting data repository of source data and artifacts produced. The raw data was pushed to a structured location and products (metrics, confusion matrix, feature metadata and prediction exports) were stored in S3 as well. This offers reproducibility: when computing nodes are rebooted or shut down, artifacts are not lost, and can be versioned. S3 also facilitates lifecycle and policies on access which come in handy when storage is to be expanded with time. [15][16]

Modeling layer:

The development of the models was done in SageMaker notebook compute. The loading of data, data split, training and evaluation were carried out using code using deterministic random seeds. This notebook-based stage had traceable results and allowed conducting an experiment iteratively. Though an endpoint that is managed real-time was originally intended to be deployed, the modeling layer was still valid and complete regardless of whether this endpoint was serving in mode. [5]

Serving layer:

The targeted initial directing mode was SageMaker real-time endpoint of managed model hosting. Nevertheless, with sandbox restrictions the endpoint provisioning attempt was not successful and resources were rolled back. A serverless inference path was used to maintain delivery and prove it to be operationally viable. In this fallback architecture, Lambda is the inference function and gets prediction artifacts which have been precomputed in S3. This is not a complete online model inference but does maintain API behavior, can be combined with cloud controls and exhibits production-relevant service composition. [5], [6]

API access layer:

API Gateway provides a clean HTTP route, and is not tied to the logic of the backends. The interface is explicit and testable as it determines the route design and stage deployment. This additionally facilitates the future hardening measures like authorizers, usage plans, throttling and custom domains. Notably, API Gateway offers independent telemetry to Lambda, which allows to better differentiate between transport-level and compute-level problems. [7]

Operations layer:

CloudWatch is able to offer observability along the runtime path. In the case of Lambda, Invocations, Duration and Errors were measured. In the case of API Gateway, the Latency, 4XX and 5XX were measured. These cloud KPIs played a critical role in assignment evidences and are also critical in production where the quality of the models is inadequate without service reliability and performance visibility. [8][18]

Governance and security issues:

The IAM permissions were limited to the necessary access, in particular, S3 read permissions on artifacts and execution trace logging permissions. Stricter least-privilege controls, KMS-based encryption, and VPC-bound execution can be extended to the design to provide even stricter control in contexts. Moreover, based on the integration requirements API Gateway can be secured with the JWT authorizers or API keys. Architectural rationale. The last architecture strikes the correctness, feasibility and transparency with constraints. It maintains the fundamental lifecycle data in S3, model

in notebook workflow, service via API, telemetry via CloudWatch, and cleanup discipline to manage cost. It proves that strong AI ecosystem delivery is not only a matter of architecture and operations as it is a matter of choice of an algorithm. The environment in which it is implemented: AWS Academy sandbox in eu-north-1 (Europe, Stockholm). Service selections were made to be sandbox friendly and preserve architectural purpose. [10], [11]

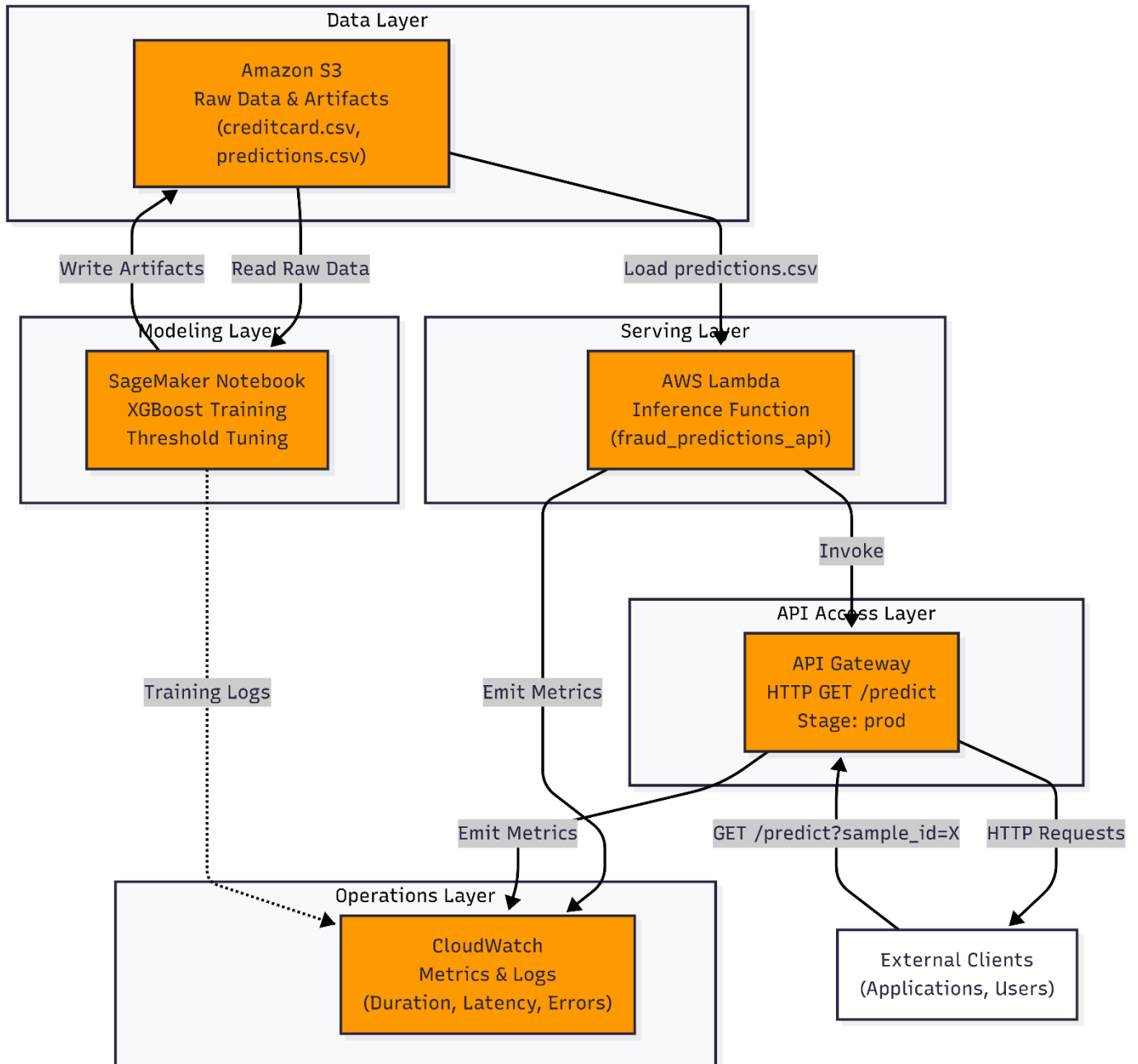


Figure 1: Architecture used to create the fraud risk scoring AI ecosystem on AWS (training in SageMaker notebook, serving via Lambda/API Gateway, monitoring via CloudWatch).

Model Description

The ML element was developed as a monitored two-classifier with XGBoost which was selected due to its high tabular results, processor efficiency, as well as its ability to deal with and compute non-linear correlations. The data set consists of artificially created features and binary target variable of fraudulent and legitimate transactions. A train test split was stratified to ensure that the split preserved partitions as far as the class proportions are concerned. [1]

Class imbalance handling:

Since there are limited cases of frauds, it had to be treated unequally. The training flow achieved scale pos weight based on the count of classes in the training split and provided it to xgboost. This optimizes with a unity of minority classification, enhancing the sensitivity of fraud, without unnecessarily maximizing the aggregate accuracy. [2]

Training configuration:

The model was optimized on hyperparameter settings such as the number of estimators, tree depth, the learning rate and subsampling parameters. Probabilistic outputs were originally evaluated, and calibration with threshold taking place afterwards. A sweep over thresholds was also used instead of using a default threshold of 0.5, to determine an operating point in which F1 balance would optimally detect workloads in relation to fraud detection. Calibration of threshold and KPIs. The threshold considered was 0.87. At this operating point, the model achieved: Precision 0.9302, Recall 0.8163, F1 0.8697, ROC-AUC 0.9774, and PR-AUC 0.8778. Such values reflect a high level of discrimination and realistic performance of minorities. High precision can also prove valuable in diminishing the amount of unwarranted intervention workload, and recall is robust enough to capture fraud that is significant. [2], [3]

Artifacts and reproducibility:

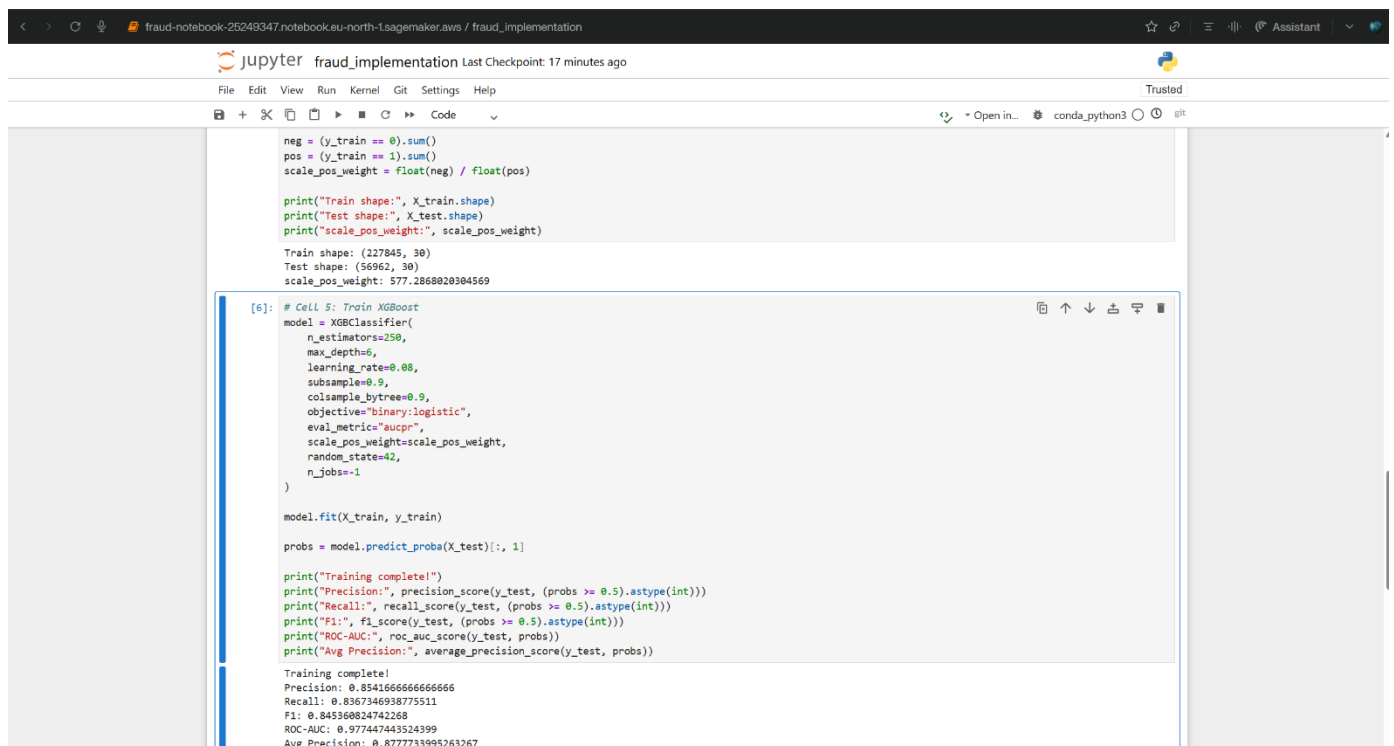
The implementation also exported an entire bundle of artifacts, including trained model file, metrics JSON, feature metadata, confusion matrix CSV and classification report CSV. Other prediction outputs were exported and put in S3 to be served in the fallback runtime path. The artifact discipline facilitates consistency, auditing, and traceable transition of development to deployment phases of development.

Deployment adaptation:

The initial endpoint to be deployed was a managed SageMaker endpoint. Attempts to provision were made, which were unsuccessful because sandbox restrictions existed coupled with authorized instance restrictions. Instead of having to leave deployment as it was, the project moved to a serverless serving model with Lambda and API Gateway with S3-based prediction lookup. This offered an operational API that delivered the probability and label outputs of frauds, and KPIs of clouds that were measured. Mitigation and challenges of implementation. Environment constraints, endpoint creation delays/failures and setting the scales between model and service-centric evidence requirements seemed to be the key challenges. Mitigations were to save all model outputs prior to deployment attempts, deterministic export of artifacts, falling back deployment path; and full screenshots of runtime that were to be used to validate. [5]-[7]

Overall model assessment:

The modeling phase had a high level of quantitative performance and became a part of a functional cloud service journey. Though a full-fledged endpoint would be a better choice in an unrestricted setting, the adopted fallback nonetheless displays an end-to-end AI ecosystem with verifiable results and justifiable engineering choices. Attribution: The author used standard Python libraries as well as AWS services to implement modeling. Architectural guidelines and service configuration were acquired with the aid of AWS documentation and course materials. Some of the execution evidences were recorded as screenshots: people illustrating the results of the model training, threshold tuning, and the successful invocation of Lambda, the successful answer of the API, CloudWatch KPI charts, and listing of the artifacts stored on S3.



```

neg = (y_train == 0).sum()
pos = (y_train == 1).sum()
scale_pos_weight = float(neg) / float(pos)

print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)
print("scale_pos_weight:", scale_pos_weight)

Train shape: (227845, 30)
Test shape: (56962, 30)
scale_pos_weight: 577.2868020304569

[6]: # Cell 5: Train XGBoost
model = XGBClassifier(
    n_estimators=250,
    max_depth=6,
    learning_rate=0.08,
    subsample=0.9,
    colsample_bytree=0.9,
    objective="binary:logistic",
    eval_metric="aucpr",
    scale_pos_weight=scale_pos_weight,
    random_state=42,
    n_jobs=-1
)

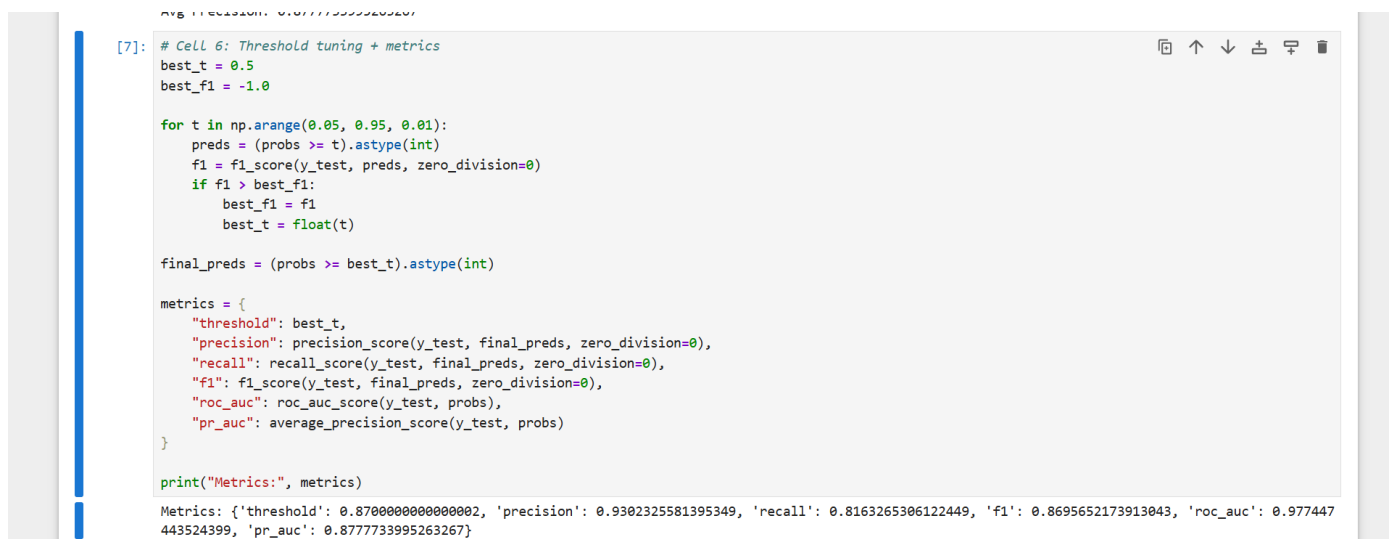
model.fit(X_train, y_train)

probs = model.predict_proba(X_test)[:, 1]

print("Training complete!")
print("Precision:", precision_score(y_test, (probs >= 0.5).astype(int)))
print("Recall:", recall_score(y_test, (probs >= 0.5).astype(int)))
print("F1:", f1_score(y_test, (probs >= 0.5).astype(int)))
print("ROC-AUC:", roc_auc_score(y_test, probs))
print("Avg Precision:", average_precision_score(y_test, probs))

Training complete!
Precision: 0.8541666666666666
Recall: 0.8367346938775511
F1: 0.845360824742268
ROC-AUC: 0.977447443524399
Avg Precision: 0.8777733995263267
    
```

Figure 2: Model Training Output



```

[7]: # Cell 6: Threshold tuning + metrics
best_t = 0.5
best_f1 = -1.0

for t in np.arange(0.05, 0.95, 0.01):
    preds = (probs >= t).astype(int)
    f1 = f1_score(y_test, preds, zero_division=0)
    if f1 > best_f1:
        best_f1 = f1
        best_t = float(t)

final_preds = (probs >= best_t).astype(int)

metrics = {
    "threshold": best_t,
    "precision": precision_score(y_test, final_preds, zero_division=0),
    "recall": recall_score(y_test, final_preds, zero_division=0),
    "f1": f1_score(y_test, final_preds, zero_division=0),
    "roc_auc": roc_auc_score(y_test, probs),
    "pr_auc": average_precision_score(y_test, probs)
}

print("Metrics:", metrics)

Metrics: {'threshold': 0.8700000000000002, 'precision': 0.9302325581395349, 'recall': 0.8163265306122449, 'f1': 0.8695652173913043, 'roc_auc': 0.977447443524399, 'pr_auc': 0.8777733995263267}
    
```

Figure 3: Threshold Tuning

Serverless Credit Card Fraud Risk Scoring on AWS: Design, Deployment, and Scalability Considerations

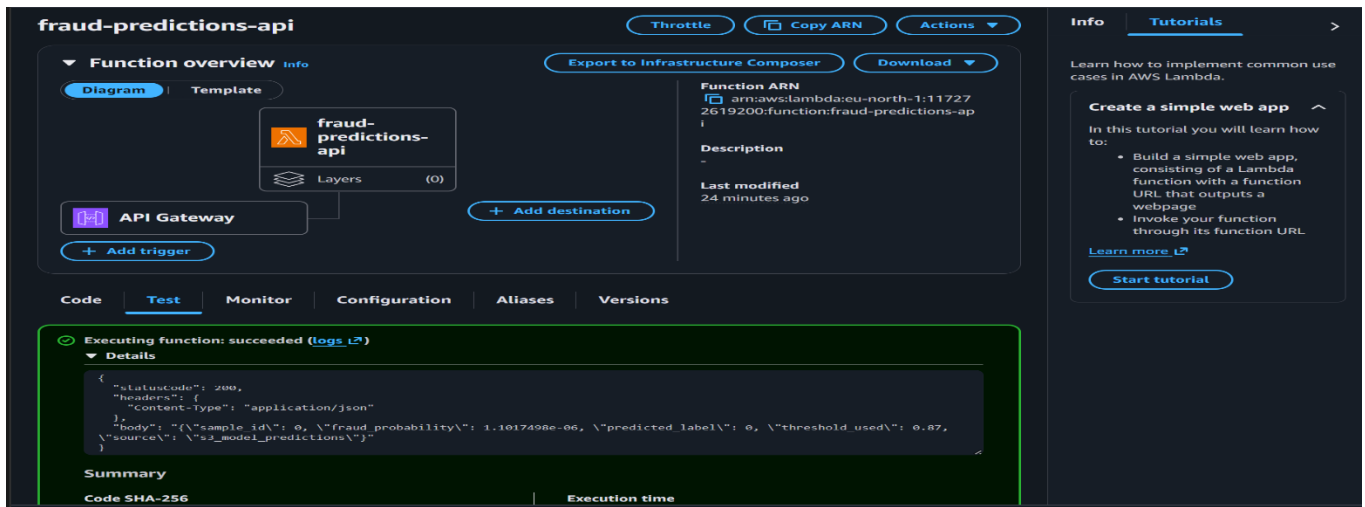


Figure 4: Lambda Test Result

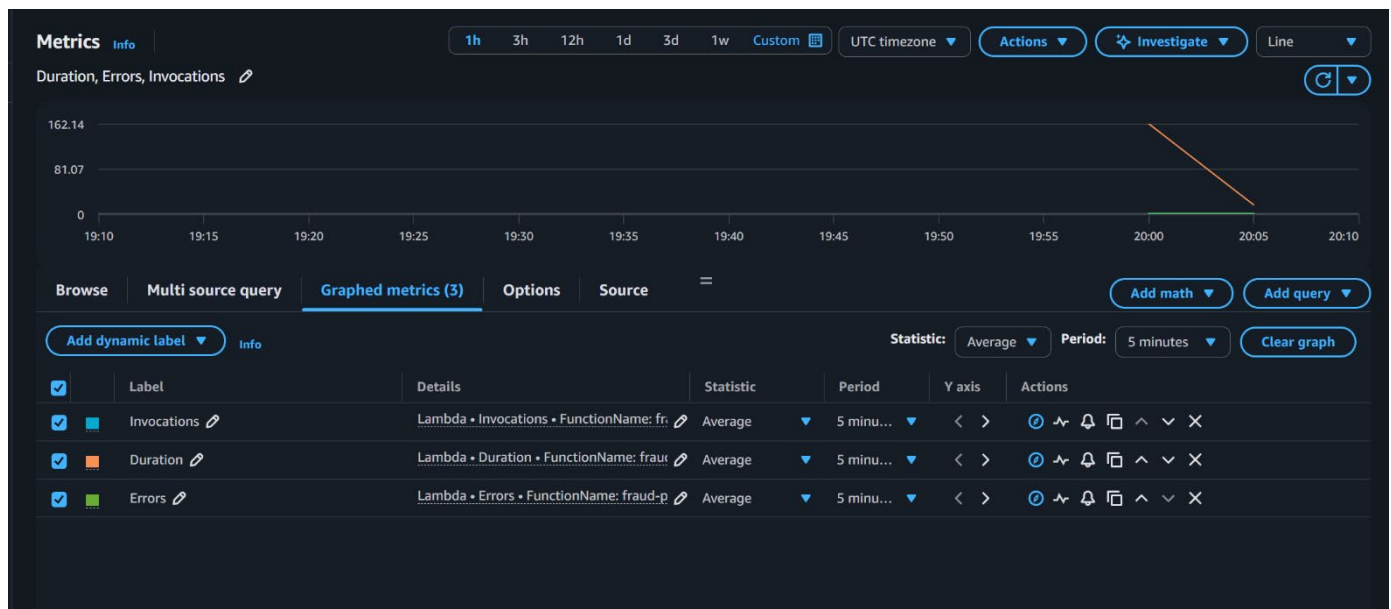


Figure 5: API Lambda Metrics

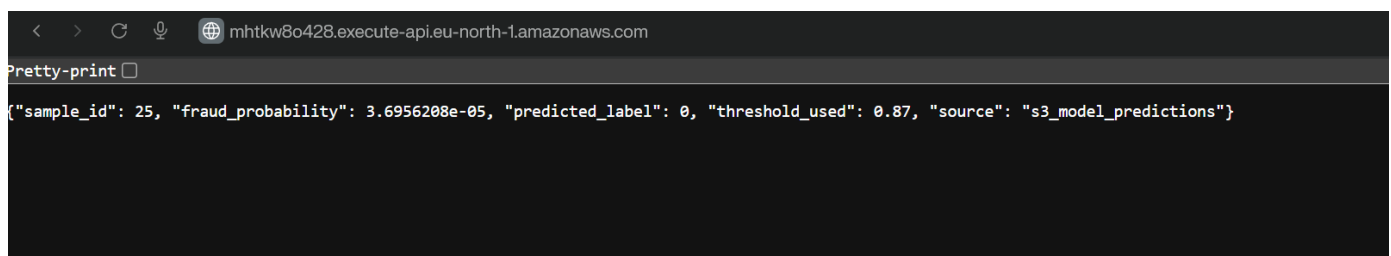


Figure 6: API Response

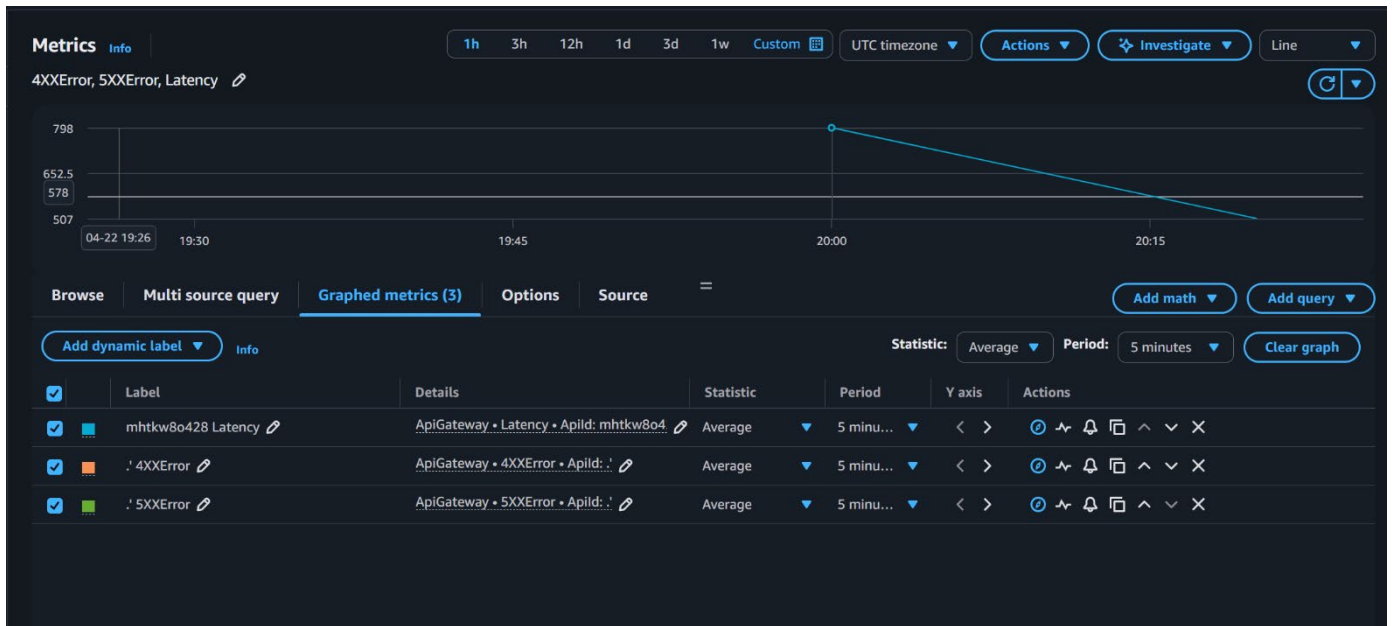


Figure 7: Cloud Watch API metrics

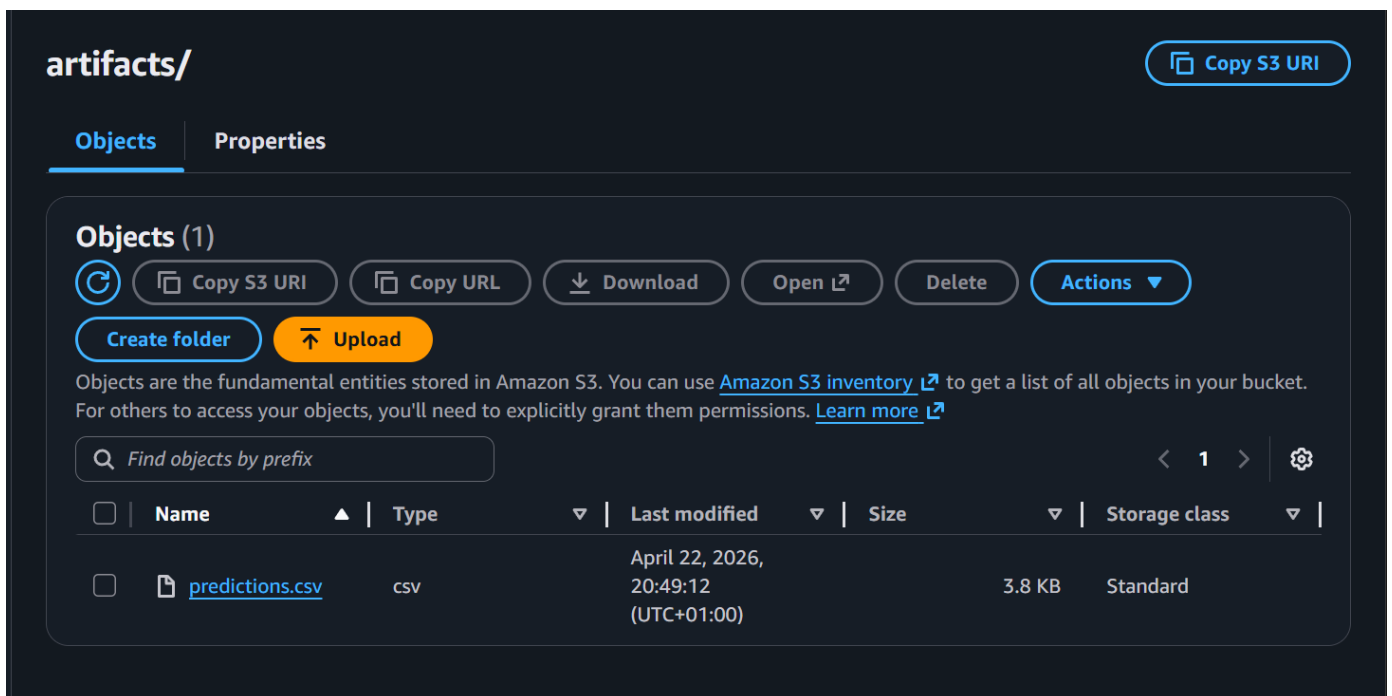


Figure 8: S3 Artifact listing

Scalability Considerations

Scalability planning is dealt with on architectural, operational, as well as governance. Its current implementation is deliberately sparse however can scale up with the number of requests, the speed of data delivery and the scale of the business. [9] [18]

API gateway and Lambda can also scale horizontally at larger traffic levels and reserved and provided concurrency can be used to control concurrency limits and cold start spikes to achieve consistent latency. To sustain real-time scoring, a SageMaker endpoint, which is reintroduced based on invocation and latency-based policies should be used with autoscaling. [5]- [7]

To minimize rollback risk, reliability needs to be enhanced with retries, idempotency, dead-letter queues, and blue/green model deployments. CloudWatch incident workflows, health checks, and alarms enhance the detection and recovery time. [8], [9]

There should be the maturity of security in terms of tighter IAM least-privilege roles, encrypted-at-rest and encrypted-at-transit, stricter controls on the API (authorizers, validation, throttling), and audit visibility in a central place via CloudTrail and logs. Serverless efficiency monitoring, Cost Explorer, Budgets, Trusted Advisor and S3 lifecycle policies should be used as a control over cost. A right-sized managed inference can reduce the unit cost, in case the demand is steady and high-throughput. [10]- [14]

CI/CD gates are needed to package, validate, and deploy models- operate CI/CD, triage, rollback, evidence capture runbooks are needed. Operational and statistical drift need to be tracked simultaneously in observability to identify both platform KPIs and model KPIs. Fraud activity is dynamic, resulting in the need to schedule retraining, data quality checks, data drift monitoring, as well as reproducible pipelines when it comes to long-term performance. [9]

The sustainability must still focus on cleanup discipline, right-sizing and autoscaling instead of having idle always-on infrastructure. In general, this roadmap is in line with all the AWS Well-Architected pillars: Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, and Sustainability. [9], [18]

Conclusion: the solution is both scalable and viable within the bounds of current limitations and the suggested controls offers a clear roadmap to production-grade scaled model without compromising reliability, security, cost discipline or model performance. [9]

References

- [1] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD '16), San Francisco, CA, USA, Aug. 2016, pp. 785-794, doi: 10.1145/2939672.2939785. [Online]. Available: <https://doi.org/10.1145/2939672.2939785>
- [2] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, "Calibrating Probability with Undersampling for Unbalanced Classification," in 2015 IEEE Symp. Series on Computational Intelligence (SSCI), Cape Town, South Africa, Dec. 2015, pp. 159-166, doi: 10.1109/SSCI.2015.33. [Online]. Available: <https://doi.org/10.1109/SSCI.2015.33>
- [3] T. Saito and M. Rehmsmeier, "The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets," PLOS ONE, vol. 10, no. 3, p. e0118432, 2015, doi: 10.1371/journal.pone.0118432. [Online]. Available: <https://doi.org/10.1371/journal.pone.0118432>
- [4] Kaggle, "Credit Card Fraud Detection." [Online]. Available: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
- [5] Amazon Web Services, "What is Amazon SageMaker?" [Online]. Available: <https://docs.aws.amazon.com/sagemaker/latest/dg/whatis.html>
- [6] Amazon Web Services, "AWS Lambda Developer Guide." [Online]. Available: <https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>
- [7] Amazon Web Services, "Amazon API Gateway Developer Guide." [Online]. Available: <https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html>
- [8] Amazon Web Services, "What is Amazon CloudWatch?" [Online]. Available: <https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.html>
- [9] Amazon Web Services, "AWS Well-Architected Framework." [Online]. Available: <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>
- [10] Amazon Web Services, "IAM Security Best Practices." [Online]. Available: <https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html>
- [11] Amazon Web Services, "AWS CloudTrail User Guide." [Online]. Available: <https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-user-guide.html>
- [12] Amazon Web Services, "AWS Cost Explorer." [Online]. Available: <https://docs.aws.amazon.com/cost-management/latest/userguide/ce-what-is.html>
- [13] Amazon Web Services, "AWS Budgets." [Online]. Available: <https://docs.aws.amazon.com/cost-management/latest/userguide/budgets-managing-costs.html>
- [14] Amazon Web Services, "AWS Trusted Advisor." [Online]. Available: <https://docs.aws.amazon.com/awssupport/latest/user/trusted-advisor.html>
- [15] Amazon Web Services, "What is Amazon S3?" [Online]. Available: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html>

[16] P. Denny, "Week 5 - Storage and Databases," lecture notes, CS5024 - Theory and Practice of Advanced AI Ecosystems

[17] P. Denny, "Week 6 - Cloud Architecture," lecture notes, CS5024 - Theory and Practice of Advanced AI Ecosystems

[18] P. Denny, "Week 8 - Automatic Scaling and Monitoring," lecture notes, CS5024 - Theory and Practice of Advanced AI Ecosystems